

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Martin Christian Suter Pauliac

11 de septiembre de 2026

00:59

1. Introducción

En el presente informe presento el análisis de rendimiento de distintos algoritmos clásicos de ordenamiento y multiplicación de matrices. En la teoría, sabemos cuánto deberían tardar matemáticamente, pero aquí pondremos a prueba la teoría programándolos en C++ y midiendo sus tiempos reales, además del uso de memoria en el sistema.

Para el ordenamiento, evaluamos sort, QuickSort, MergeSort y PatienceSort usando arreglos de hasta diez millones de datos. Por otra parte, comparamos el método Naive versus el algoritmo de Strassen puro multiplicando matrices cuadradas. El objetivo principal de este experimento es ver cómo detalles de la teoría a veces ignoran aspectos importantes del sistema como la petición de memoria dinámica constante o cómo funciona el cache del computador, puntos esenciales que terminan afectando drásticamente la velocidad del programa.

2. Entorno experimental

Para la realización de este experimento, todos los algoritmos se programaron en C++17 y se compilieron utilizando g++ con la bandera de optimización -O3. Las pruebas fueron ejecutadas en un entorno Linux sobre un Lenovo IdeaPad Slim 3. Las especificaciones técnicas del equipo son un procesador AMD Ryzen 5 7520U (4 núcleos y 8 hilos), 8gb de memoria RAM a 5500 MHz y almacenamiento SSD NVMe.

Un detalle clave para entender los resultados es la memoria cache del procesador, que está dividida en L1 (256 KB), L2 (2 MB) y L3 (4 MB). Tener claros estos límites físicos del hardware nos servirá para

explicar mas adelante por que ciertos algoritmos que iteran de forma lineal (como el metodo clasico de las matrices) logran tiempos tan rapidos al aprovechar bien el espacio de la cache, en comparacion con los algoritmos que terminan ahogando la memoria RAM por estar pidiendo espacio dinamico constantemente.

En cuanto a los datos de prueba, en el ordenamiento usamos arreglos desde 10 hasta 10^7 elementos, usando datos aleatorios, ascendentes y descendentes. Por otra parte para las matrices ocupamos dimensiones cuadradas partiendo de 16×16 hasta 1024×1024 . Finalmente, para evitar que algun proceso del sistema intervenga en las mediciones sobre el tiempo de CPU, ejecutamos cada configuracion tres veces seguidas y utilizamos el promedio de esos intentos para construir los graficos logaritmicos.

3. Resultados y Análisis

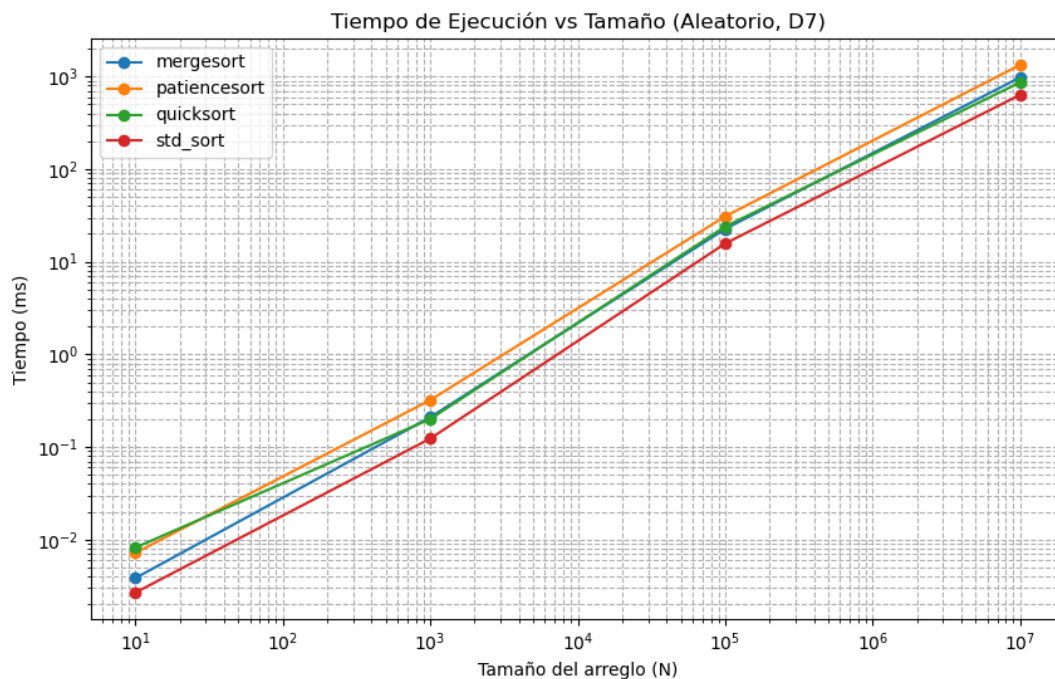


Figura 1: Tiempos de ejecución empiricos para los algoritmos de ordenamiento (Caso Aleatorio, D7).

En la figura correspondiente al ordenamiento de arreglos aleatorios se observa que todos los algoritmos presentan una pendiente similar en la escala logarítmica confirmando graficamente su complejidad teorica de $O(n \log n)$. Sin embargo, sort domina consistentemente en todos los tamaños de arreglos ocupados. Esto se debe a la naturaleza hibrida que este posee, ademas de un buen uso de la memoria cache del procesador. QuickSort y MergeSort muestran un desempeño parejo, en arreglos de gran magnitud de datos practicamente se superponen entre si. Por otra parte, PatienceSort a pesar de tener un respaldo teorico positivo es el algoritmo que tiene mayor tiempo de ejecución, probablemente debido a el gran costo

computacional que genera la administracion constante de estructuras dinamicas y colas de prioridad.

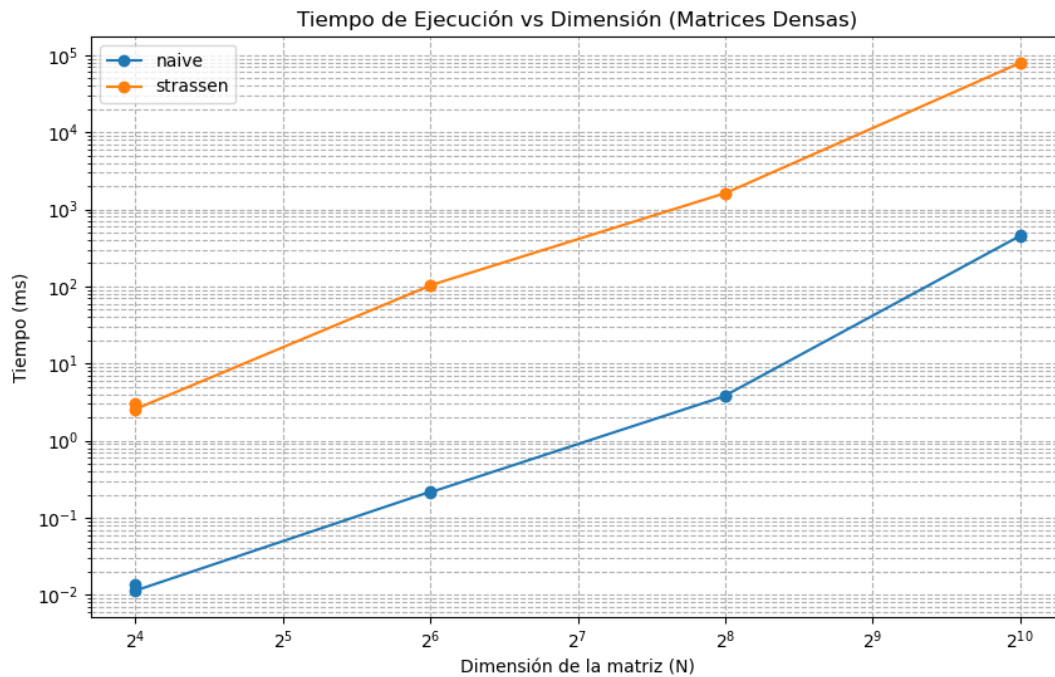


Figura 2: Tiempos de ejecucion empiricos para la multiplicacion de matrices (Caso Denso).

En el analisis de las multiplicaciones de matrices densas, los resultados empiricos contrastan drásticamente con la expectativa teorica. En la teoria Strassen posee una cota asintotica superior al algoritmo Naive, sin embargo la grafica demuestra que la implementacion pura de strassen es constantemente mas lenta que el algoritmo Naive por varios ordenes de magnitud, alcanzando tiempos absurdamente lentos para las matrices de 1024. Esta baja de velocidad masiva se debe probablemente a las constantes ocultas de su recursion pura. Al dividir el problema hasta el caso base de 1×1 , Strassen exige la instanciacion dinamica y copia millones de sub-vectores temporales en la RAM, esto provoca una saturacion en el sistema ralentizando el proceso considerablemente. Por el contrario, la implementacion Naive, optimizada en su bucle interno maximiza al iterar linealmente sobre la memoria.

Al revisar los datos de memoria RAM en los resultados, notamos que para arreglos grandes, casi todos los algoritmos parecen consumir exactamente la misma cantidad de memoria (alrededor de 600 MB para arreglos de 10 millones). Sin embargo, esto ocurre por la forma en que el sistema mide los datos. La funcion de C++ que utilizamos no mide el consumo en cada milisegundo, sino que guarda el tope maximo de todo el programa. Como el codigo ejecuta los algoritmos uno detras de otro, los metodos que son muy eficientes con la memoria (sort o QuickSort) son "tapados" por la medicion mas grande registrada previamente. El principal responsable de esto es el PatienceSort y strassen en caso de las matrices, ya que constantemente piden al sistema crear miles de arreglos pequenos y estructuras dinamicas para poder funcionar. Aunque en la tabla de datos los numeros de RAM se ven repetidos por el mayor dato registrado,

la prueba real de que la memoria colapso se ve en los graficos de tiempo, PatienceSort y strassen terminan siendo mucho mas lentos que sus contrapartes por todo el esfuerzo extra del sistema manejando la memoria pedida.

4. Conclusiones

Lo mas relevante que deja este analisis experimental es comprobar que la cota matematica sirve como una gran guia teorica, pero aplicado al momento de programar no es lo unico que define la velocidad y uso de memoria de un programa. Cosas que en papel pueden parecer detalles irrelevantes o de poco impacto, como pedir constantemente memoria dinamica o aprovechar al maximo la cache, son las que en realidad definen si un programa sera rapido o lento en la practica. El caso del algoritmo de strassen es el mejor ejemplo de esto, este termino siendo casi 200 veces mas lento que el metodo clasico iterativo en matrices de tamaño 1024×1024 , esto debido al enorme costo que presenta la recursion profunda en la memoria del computador. Con esto queda claro que los algoritmos complejos como strassen para adaptarse a la practica deben tener casos bases hibridos, como saltar a Naive en contextos especificos, logrando evitar la saturacion de la RAM del sistema operativo.

Referencias

- [1] GeeksforGeeks. *Patience Sorting*. Disponible en: <https://www.geeksforgeeks.org/dsa/patience-sorting/> Recurso utilizado para comprender la logica de formacion de las pilas (piles) y la extraccion del elemento minimo utilizando colas de prioridad (Min-Heap).
- [2] GeeksforGeeks. *Merge Sort Algorithm*. Disponible en: <https://www.geeksforgeeks.org/dsa/merge-sort/> Utilizado como referencia base para la implementacion optima de la funcion de mezcla (merge) y la estrategia de divide y vencerás sobre arreglos.
- [3] GeeksforGeeks. *QuickSort Algorithm*. Disponible en: <https://www.geeksforgeeks.org/dsa/quick-sort-algorithm/> Referencia para la logica de particion del arreglo y la seleccion del pivote aleatorio para evitar caer en el peor caso teorico.
- [4] GeeksforGeeks. *Strassen's Matrix Multiplication*. Disponible en: <https://www.geeksforgeeks.org/dsa/strassens-matrix-multiplication/> Guia teorica ocupada para estructurar las 7 formulas de multiplicacion de sub-matrices y las operaciones de suma/resta necesarias para ensamblar la matriz resultante.
- [5] GeeksforGeeks. *Matrix Multiplication*. Disponible en: <https://www.geeksforgeeks.org/matrix-multiplication/> Referencia para la implementacion del algoritmo tradicional iterativo (Naive) de complejidad $O(n^3)$ enfocado en la maximizacion de la localidad espacial.

5. Implementaciones y Código Fuente

El link para el repositorio donde esta la tarea:

<https://github.com/MartinSuter/Tarea-1-Algoritmos-y-Complejidad>